Université Mohammed V de Rabat
École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Deep Learning – Lecture 8

Réseaux récurrents : RNN, LSTM et GRU

Pr. Aissam BERRAHOU

ENSIAS – Université Mohammed V de Rabat

Module Deep Learning – 2026-2027

Objectifs de la lecture

- caractériser une donnée séquentielle et préparer ses tenseurs (B, T, d) ;
- écrire et dérouler les équations d'un RNN simple ;
- distinguer les configurations un-à-plusieurs, plusieurs-à-un, plusieurs-à-plusieurs ;
- expliquer la BPTT et la disparition/explosion du gradient ;
- écrire les équations LSTM et GRU et dénombrer leurs paramètres ;
- concevoir des RNN empilés, bidirectionnels, encodeur-décodeur avec Keras.

Plan de la lecture

1. Données séquentielles
2. Le RNN simple
3. Rétropropagation dans le temps
4. LSTM
5. GRU et architectures

Plan

1. Données séquentielles

2. Le RNN simple

3. Rétropropagation dans le temps

4. LSTM

5. GRU et architectures

Pourquoi des réseaux récurrents ?

Séquence

$\mathbf{x}_1, \ldots, \mathbf{x}_T$ ordonnée, T variable : texte, parole, séries temporelles, vidéo, ADN.

Limites du MLP/CNN

- taille d'entrée fixe ;
- pas de partage dans le temps ;
- pas de mémoire longue.

Dépendance lointaine

« Le chat que j'ai adopté l'an dernier *dort* » : accord à 7 mots de distance.

Plan

1. Données séquentielles

2. Le RNN simple

3. Rétropropagation dans le temps

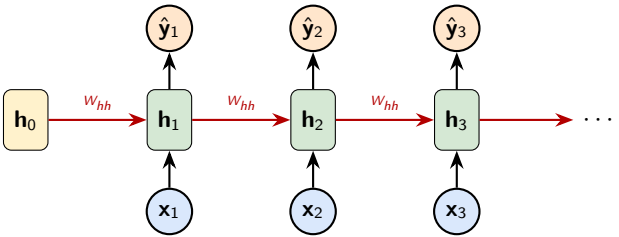
4. LSTM

5. GRU et architectures

Cellule récurrente et dépliement

Équations (Elman)

$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \qquad \hat{\mathbf{y}}_t = g(W_{hy}\mathbf{h}_t + \mathbf{b}_y)$$



Partage des poids

Paramètres : $n(d + n + 1)$, indépendants de T. Ex. $d = 50$, $n = 64$: 7 360.

Exemple numérique

$d = n = 1,\ w_{xh} = 0{,}5,\ w_{hh} = 0{,}8,\ h_0 = 0,\ x = (1;\ 0{,}5;\ -1)$

Propagation avant

$h_1 = \tanh(0{,}5) = 0{,}462$
$h_2 = \tanh(0{,}25 + 0{,}370) = 0{,}551$
$h_3 = \tanh(-0{,}5 + 0{,}441) = -0{,}059$

La mémoire des entrées positives compense en partie $x_3 < 0$.

Pourquoi tanh ?

Borne l'état dans $]-1, 1[$; ReLU dans la récurrence peut faire exploser $\mathbf{h}_t$.

Configurations

Configuration	Applications
Un-à-plusieurs	légende d'image, génération de musique
Plusieurs-à-un	sentiment, classification de séries, prévision
Plusieurs-à-plusieurs (décalé)	traduction, résumé (encodeur-décodeur)
Plusieurs-à-plusieurs (synchrone)	étiquetage grammatical, entités nommées

Keras

`return_sequences=True` : renvoie $(\mathbf{h}_1, \ldots, \mathbf{h}_T)$; nécessaire pour empiler ou pour une sortie synchrone.

Plan

1. Données séquentielles

2. Le RNN simple

3. Rétropropagation dans le temps

4. LSTM

5. GRU et architectures

BPTT et disparition du gradient

BPTT

Rétropropagation sur le réseau déplié ; gradients des poids partagés **sommés** sur t :

$$\frac{\partial J}{\partial W_{hh}} = \sum_t \boldsymbol{\delta}_t^h \mathbf{h}_{t-1}^T, \quad \frac{\partial J}{\partial \mathbf{h}_t} = W_{hy}^T \boldsymbol{\delta}_t^y + W_{hh}^T \boldsymbol{\delta}_{t+1}^h$$

Produit de jacobiennes

$\partial h_3 / \partial h_1 = 0{,}8 \cdot 0{,}997 \cdot 0{,}8 \cdot 0{,}696 \approx 0{,}44$; sur 50 pas avec facteur 0,6 : $0{,}6^{50} \approx 8 \times 10^{-12}$.

Explosion

Écrêtage : $\mathbf{g} \leftarrow \tau \mathbf{g} / \|\mathbf{g}\|$ si $\|\mathbf{g}\| > \tau$.

Disparition

Mémoire courte (10 pas) $\Rightarrow$ cellules à portes.

Préparer les données

Séries temporelles : fenêtrage

$(10, 12, 13, 15, 16, 18)$, fenêtre 3 :
$(10, 12, 13) \to 15$, $(12, 13, 15) \to 16$,
$(13, 15, 16) \to 18$

Découpage **chronologique** ; normalisation sur l'entraînement seul.

Texte

Tokenisation $\to$ remplissage à $T \to$ `Embedding`.
« ce film est très bon » $\to (1, 2, 3, 4, 5, 0) \to$ matrice 6×8.

Plan

1. Données séquentielles

2. Le RNN simple

3. Rétropropagation dans le temps

4. LSTM

5. GRU et architectures

La cellule LSTM

Équations

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f) \quad \text{oubli}$$
$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i) \quad \text{entrée}$$
$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{candidat}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \textbf{additif}$$
$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o) \quad \text{sortie}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Pourquoi ça marche

$\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx \mathrm{diag}(\mathbf{f}_t)$: si $\mathbf{f} \approx 1$, le gradient traverse le « tapis roulant ».
$0{,}95^{50} \approx 0{,}077$ vs $0{,}6^{50} \approx 10^{-11}$.

Paramètres

$4n(d + n + 1)$; biais d'oubli initialisé à 1.

Un pas de LSTM

Pré-activations $a_f = 2$, $a_i = 0$, $a_c = 1$, $a_o = 1$, $c_{t-1} = 0{,}8$

Calcul

$f_t = 0{,}881, \quad i_t = 0{,}5, \quad \tilde{c}_t = 0{,}762, \quad o_t = 0{,}731$
$c_t = 0{,}881 \times 0{,}8 + 0{,}5 \times 0{,}762 = 1{,}086$
$h_t = 0{,}731 \times \tanh(1{,}086) = 0{,}582$

Avec $a_f = -3$: $f_t = 0{,}047$, $c_t = 0{,}419$: la mémoire est presque effacée.

Plan

1. Données séquentielles

2. Le RNN simple

3. Rétropropagation dans le temps

4. LSTM

5. GRU et architectures

La cellule GRU

Deux portes, un seul état

$\mathbf{z}_t = \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z)$ — mise à jour
$\mathbf{r}_t = \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r)$ — réinitialisation
$\tilde{\mathbf{h}}_t = \tanh(W_h \mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$
$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$

($d = 32$, $n = 64$)	**Portes**	**États**	**Paramètres**
SimpleRNN	0	$\mathbf{h}$	6 208
LSTM	3	$\mathbf{h}, \mathbf{c}$	24 832
GRU	2	$\mathbf{h}$	18 624

Empilés, bidirectionnels, encodeur-décodeur

Empilés

Séquence d'états de la couche l = entrée de $l+1$; `return_sequences=True`.

Bidirectionnels

$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t ; \overleftarrow{\mathbf{h}}_t]$: contexte passé et futur (pas pour la prévision temps réel).

Encodeur-décodeur

L'encodeur résume, le décodeur génère mot à mot.

Goulot d'étranglement

Un seul vecteur de contexte pour toute la phrase $\Rightarrow$ **attention** (L9).

Keras

```
model = tf.keras.Sequential([
    layers.Input(shape=(200,)),
    layers.Embedding(10000, 64, mask_zero=True),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True, dropout=0.2)),
    layers.Bidirectional(layers.LSTM(32)),
    layers.Dense(1, activation='sigmoid')])

prevision = tf.keras.Sequential([layers.Input(shape=(48, 1)),
    layers.GRU(64, return_sequences=True), layers.GRU(32), layers.Dense(12)])
prevision.compile(optimizer=tf.keras.optimizers.Adam(clipnorm=1.0), loss='mse')
```

À retenir

- RNN : état caché = mémoire, poids partagés, BPTT.
- Gradient qui disparaît $\Rightarrow$ LSTM/GRU (mise à jour **additive** ou interpolation).
- Bonnes pratiques : `clipnorm`, `dropout`, `mask_zero`, `EarlyStopping`.

Questions de révision

1. Paramètres de `SimpleRNN(128)` sur des plongements de 100 ? (29 312)
2. Forme de sortie de `Bidirectional(LSTM(64, return_sequences=True))` pour $(32, 200, 64)$?
3. GRU avec $\mathbf{z} = \mathbf{0}$: que se passe-t-il ?